# FastDedup

## A fast and memory efficient tool for reads deduplication

**Raphaël Ribes**, **Céline Mandier**
*Université de Montpellier*

24 March 2026

**Abstract** – TODO

# Table of contents

1 – Context ..... 3
2 – Existing tools and limitations ..... 4
2.1 : Fastp ..... 4
2.2 : FastUniq ..... 4
2.3 : Clumpify ..... 4
2.4 : SeqKit ..... 5
2.5 : CD-HIT-DUP ..... 5
2.6 : PRINSEQ++ ..... 6
2.7 : SAM/BAM ecosystem tools ..... 6
3 – FastDedup: High-Performance FASTX Deduplication ..... 7
3.1 : Algorithmic Complexity and Hardware-Level Optimizations ..... 7
3.2 : The Mathematics of Dynamic Hashing ..... 8
3.3 : FxHash: Algorithmic Mechanics and ALU Optimizations ..... 9
3.4 : Architecture ..... 11
4 – Benchmarking and performance evaluation ..... 12
4.1 : Compressed Paired-End FASTQ ..... 12
4.2 : Uncompressed Paired-End FASTQ ..... 13
4.3 : Compressed in, decompressed out Paired-End FASTQ ..... 13
4.4 : Compressed Single-End FASTQ ..... 14
4.5 : Decompressed Single-End FASTQ ..... 15
4.6 : Compressed in, decompressed out Single-End FASTQ ..... 15
5 – Discussion and future perspectives ..... 15
5.1 : Output Quality and Deduplication Accuracy ..... 15
5.2 : Performance with Paired-End Reads ..... 17
5.3 : Performance with Single-End Reads ..... 17
5.4 : Performance with Compressed Files ..... 17
5.5 : In short ..... 18
5.6 : The Future of FastDedup ..... 19
6 – Conclusion and Recommendations ..... 19
7 – Acknowledgements ..... 19
Bibliography ..... 20
8 – Annexes ..... 22

# 1 – Context

With the rise of New Generation Sequencing (NGS) technologies, the volume of sequencing data has increased exponentially. With this surge in data, the development of new tools for processing and analyzing sequencing reads was needed. One of the first and critical step in the analysis pipeline is the deduplication of reads, which involves identifying and removing duplicate sequences that can arise from PCR amplification or sequencing errors.

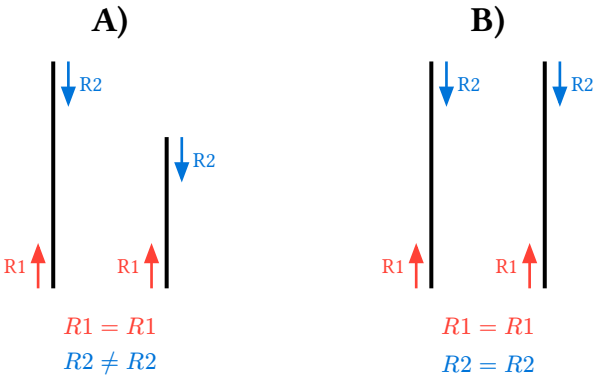



Figure 1: Identifying PCR Duplicates in Paired-End Sequencing.

**Solid Black Line:** Represents a single sequenced DNA fragment (height reflects length).

**Red Arrow (R1):** Forward read (Read 1).

**Blue Arrow (R2):** Reverse read (Read 2).

**A) Not a duplication:** Shows two distinct fragments where the start position of Read 1 is shared, but different fragment lengths lead to unique mapping positions for Read 2.

**B) PCR Duplication:** Shows two exact fragments where the start position of Read 1 and Read 2 is shared.

We define a fragment $F$, the 5′->3′ sequence as $R1$ with a length of $r_1$ and the 3′->5′ sequence as $R2$ with a length of $r_2$. We can say that $R1$ is the prefix of $F$, $R2$ reversed is the suffix of $F$ and $r_1 = r_2$.

Therefore, a PCR duplicate can be defined as if $F_1$ and $F_2$ have the same starting and ending position in the genome, with the same length. In that case, $R1_1$ and $R1_2$ are the same, and $R2_1$ and $R2_2$ are the same.

This step is crucial because removing these artifacts preventing false-positive during variant calling since PCR polymerases can introduce errors during the cycles of amplification. If an error occurs early, it is propagated into multiple daughter molecules. When sequenced, these identical, error-carrying molecules will appear as multiple independent reads supporting a mutation that does not actually exist in the biological sample [1].

Deduplication also allows correcting the allele frequency bias since not all DNA molecules amplify with equal efficiency due to factors like GC content or fragment length. This uneven amplification can artificially inflate the coverage of certain alleles. And it is also improving *de novo* assembly, especially in whole-genome sequencing. Paired-end reads are mapped to estimate the order and intervening distance between contigs, duplicates can create false-positive connections between contigs or introduce conflicting connections. [2].

Before presenting our tool, we will first review the existing tools for reads deduplication and their limitations.

# 2 – Existing tools and limitations

## 2.1 : Fastp

`fastp` is a widely adopted, all-in-one FASTQ preprocessor written in C++ that performs quality control, adapter trimming, and deduplication within a single pass [3,4]. Unlike traditional alignment-based tools that require computationally expensive mapping and coordinate sorting, `fastp` operates directly on raw sequences.

To achieve high performance, `fastp` implements a probabilistic deduplication mechanism based on Bloom filters and multiple hash arrays. For each read (or read pair), the sequence is mapped to multiple integer hash values. If all corresponding positions in the hash arrays are already marked as positive, the read is flagged as a duplicate. This streaming, single-pass architecture avoids reading and writing intermediate files to disk, drastically reducing I/O bottlenecks and resulting in ultra-fast execution speeds.

A significant advantage of `fastp` is its highly configurable memory management. Users can adjust the `--dup_calc_accuracy` parameter (ranging from level 1 to 6) to scale the hash buffer size from 1 GB up to 24 GB, allowing the tool to run on both standard personal computers and High-performance Computing (HPC) clusters.

However, this probabilistic approach involves a minor trade-off in absolute precision. `fastp` has a theoretical false-positive (hash collision) rate of approximately 0.01%, which, while negligible for most routine downstream analyses, represents a conceptual difference from exact deterministic methods. Furthermore, `fastp`'s deduplication algorithm relies heavily on the exact matching of sequence lengths and coordination regions. Consequently, applying certain quality trimming operations (such as sliding-window trimming via `--cut_front` or `--cut_tail`) prior to deduplication can interfere with the duplicate identification process, as trimmed reads may no longer perfectly match their duplicate counterparts.

## 2.2 : FastUniq

`FastUniq` [5] is a deterministic deduplication tool explicitly designed for paired-end (PE) short reads. By directly comparing the nucleotide sequences of read pairs, it provides an exact identification of PCR duplicates.

Algorithmically, `FastUniq` operates through a strict three-step pipeline:
1. loads the entirety of the paired reads into memory
2. sorts them based on their sequences using a merge sort algorithm
3. identifies duplicates by sequentially comparing adjacent read pairs in the sorted list.

A notable feature of its comparison logic is its ability to handle reads of varying lengths: a shorter sequence is considered identical to a longer one if it perfectly matches its 5′ prefix. If duplicates are found, the pair containing the longest reads is retained.

While `FastUniq` guarantees absolute precision for exact duplicate removal (zero mismatches allowed), its architectural design introduces critical resource bottlenecks. Because it relies on importing the entire dataset into RAM simultaneously using a hierarchical storage structure, its memory footprint scales linearly with the input file size. Benchmarking studies have highlighted this as a severe limitation for modern high-throughput data; for instance, processing a 538 GB Hi-C dataset required up to 1 TB of RAM [6].

Consequently, while highly accurate, `FastUniq` is often impractical for massive datasets without access to HPC clusters. Furthermore, its scope is strictly limited to paired-end data, it does not tolerate sequence mismatches, and its deduplication output is known to be sensitive to the order of the input files.

## 2.3 : Clumpify

`Clumpify`, a component of the Java-based BBTools suite [7], introduces a distinct algorithmic paradigm to duplicate removal based on $k$-mer grouping. Rather than relying on exact sequence hashing or global

coordinate sorting, `Clumpify` identifies reads that share common $k$-mers and groups them into localized clusters or "clumps".

This spatial reorganization provides a unique secondary benefit. By physically placing highly similar sequences adjacent to one another in the output file, `Clumpify` dramatically improves standard `gzip` compression efficiency, as the compression algorithm can utilize much closer reference pointers.

A major functional advantage of `Clumpify` is its high sensitivity to optical duplicates. By parsing the flowcell tile and pixel coordinates embedded within Illumina FASTQ headers, `Clumpify` (via parameters such as `dupedist`) can explicitly distinguish between amplification-derived PCR duplicates and sensor-derived optical duplicates. For paired-end data, `Clumpify` predominantly uses the first read (R1) to determine the clump, allowing the paired read (R2) to passively follow its counterpart, which maintains file synchronization while minimizing computational overhead.

To manage memory, `Clumpify` implements a multi-phase strategy (KmerSplit and KmerSort). If the dataset exceeds available RAM, the tool can divide the data into temporary groups on the disk, sorting them individually before merging.

However, benchmarking shows that its reliance on the Java Virtual Machine (JVM) can lead to variable stability and high memory overhead on massive datasets. Sometimes it requires multi-threading just to maintain stability on large inputs like 500+ GB Hi-C datasets [6].

## 2.4 : SeqKit

`SeqKit` [8,9] is a comprehensive, cross-platform, and ultra-fast toolkit for FASTA and FASTQ file manipulation. Written in the Go, it benefits from efficient concurrency management and the ability to produce static binaries with no external dependencies, making it highly accessible and stable across different operating systems.

For PCR deduplication, SeqKit provides the `rmdup` subcommand. This tool relies on an exact sequence matching approach using a hashing strategy. To process massive datasets without exhausting system memory, `seqkit rmdup` offers an MD5 mode (invoked via the `-m` or `--md5` flag). In this mode, the algorithm computes and stores only the MD5 digest of each sequence in the RAM rather than the full nucleotide sequence. This extreme memory efficiency allows SeqKit to maintain a very low resource footprint, even outperforming several newer tools when handling vast amounts of single-end (SE) data.

Despite its speed and memory efficiency, `SeqKit rmdup` has a notable architectural limitation: it does not natively support the synchronized deduplication of paired-end reads. The `rmdup` command evaluates and processes individual files independently. It will check duplicates inside each file separately, so if $R1_1 = R1_2$ but $R2_1 \neq R2_2$, $R1_2$ in will be flagged as a duplicate and removed, while $R2_2$ will be retained, leading to loss of data and desynchronization of the paired files. While workarounds exist using other SeqKit subcommands (such as `seqkit pair` to resynchronize files), `SeqKit rmdup` is generally considered the optimal choice strictly for single-end libraries.

## 2.5 : CD-HIT-DUP

`CD-HIT-DUP`, a specialized utility within the broader CD-HIT package [10,11], is designed to identify and remove both identical and nearly-identical duplicates. Unlike tools restricted to exact string matching, `CD-HIT-DUP` is particulary robust against sequencing errors. It incorporates a greedy incremental clustering algorithm that tolerates a user-defined threshold of mismatches, including both substitutions and insertions/deletions.

`CD-HIT-DUP` relies on a prefix-suffix comparison approach. The tool first bins reads that share an identical prefix of $k$ nucleotides. To accelerate this binning and sorting process, it uses a dual numeric encoding strategy (initially base-5, then base-10), allowing prefixes of up to 27 nucleotides to be compactly represented as 64-bit integers. Within each bin, sequences are sorted by decreasing length, with the longest sequence acting as the cluster's seed. The algorithm then evaluates the suffixes of the remaining sequences against this

seed, utilizing a $k$-mer word-based index table to statistically predict sequence identity and bypass computationally expensive full pairwise alignments when possible. For paired-end libraries, `CD-HIT-DUP` conceptually joins the read pairs into a single sequence and checks prefixes at both ends simultaneously to ensure paired-end integrity.

While its error tolerance provides a nuanced, high-sensitivity approach to duplicate removal, `CD-HIT-DUP` introduces a severe computational bottleneck regarding memory consumption. Because the greedy clustering algorithm must retain representative sequences and cluster bins in RAM, its memory footprint scales aggressively with dataset size and the allowed mismatch threshold. Benchmarking has shown that processing a relatively modest 50 million single-end read library can consume between 33 GB and 53 GB of RAM. For massive modern datasets, such as a 538 GB Hi-C dataset, `CD-HIT-DUP` required approximately 1 TB of RAM, often crashing on resource-constrained hardware [6].

Furthermore, practical deployment of `CD-HIT-DUP` in modern automated workflows requires caution. The tool has known stability issues in containerized environments (e.g., Docker and Singularity), occasionally throwing segmentation faults or I/O assertion errors when inputs are provided via process substitution from compressed files.

## 2.6 : PRINSEQ++

`PRINSEQ++` [12] is a highly optimized C++ implementation of the Perl-based `prinseq-lite` program. It is designed to perform a comprehensive suite of quality control, filtering, trimming, and reformatting tasks for genomic and metagenomic sequence data. By utilizing POSIX threads (pthreads) for parallelization, it achieves processing speeds more than 10 to 16 times faster than its predecessor while maintaining a relatively low memory footprint.

For sequence deduplication, `PRINSEQ++` departs from exact string comparison or sorting heuristics by implementing a probabilistic data structure known as a Bloom filter. During execution, each sequence is transformed using multiple fast, non-cryptographic hash functions, and the corresponding indices in a shared bit-array are flagged. If a newly evaluated sequence maps entirely to bits that are already set to positive, it is classified as a duplicate. This probabilistic model is highly advantageous for multi-threaded environments because it enables asynchronous read and write access to the filter with minimal thread blocking.

Additionally, `PRINSEQ++` natively supports the direct reading and writing of `gzip`-compressed FASTQ and FASTA files without the need for intermediate disk I/O, which significantly reduces hard drive storage requirements. It also natively supports paired-end reads by processing pairs simultaneously and outputting synchronized R1 and R2 files.

However, the deduplication function (`-derep`) is strictly limited to exact duplicates and does not tolerate sequencing errors. Furthermore, the probabilistic nature of the Bloom filter introduces a negligible but theoretically non-zero rate of false positives. Finally, when `PRINSEQ++` is executed with multiple threads, the output sequences are not guaranteed to remain in the same order as the input files, which may require consideration if downstream tools expect strict global index synchronization.

## 2.7 : SAM/BAM ecosystem tools

In addition to tools that operate directly on raw FASTQ sequences, the bioinformatics ecosystem relies heavily on utilities designed for SAM and BAM formats. PCR deduplication tools like such as `samtools` (via `markdup`) [13] and Picard's `MarkDuplicates` [14].

While these tools are typically used downstream after mapping reads to a reference genome, it is technically possible to convert raw, unaligned FASTQ files directly into unmapped SAM/BAM files to process them through these pipelines. However, even without the computationally heavy step of genomic alignment, the sheer format conversion from FASTQ to SAM/BAM introduces significant and unnecessary I/O and processing overhead. Because our primary objective is to evaluate and develop a highly optimized, memory-efficient tool strictly for direct FASTQ-to-FASTQ deduplication, these format-dependent tools fall outside the scope of our comparative analysis.

# 3 – FastDedup: High-Performance FASTX Deduplication

The goal behind the development of `FastDedup` (or `FDedup` ; Figure 2 Figure 3) was to create a FASTX PCR deduplication tool that prioritizes maximum speed and memory efficiency. To achieve this, the tool relies on `xxh3` [15,16], a non-cryptographic hash function, to compute a unique fingerprint for each read. These fingerprints are cached in memory using `fxhash` [17], which provides a low memory footprint.

FDedup provides users with precise control over the hash collision rate while maintaining high performance. It automatically scales its hashing strategy, seamlessly switching between 64-bit and 128-bit hashes based on the estimated input sequence count and a user-defined collision probability threshold.

Specifically designed for seamless integration into OMICS pipelines, the tool features robust incremental deduplication and auto-recovery mechanisms. In the event of an interruption FDedup can safely preload existing hashes from the output file to prevent duplicate processing and smoothly resume operations. If an uncompressed output file becomes corrupted due to a crash, the tool automatically detects the issue, calculates a fail-safe truncation point, and truncates the file to the last valid sequence before continuing.

Finally, a dry-run mode (`--dry-run` or `-s`) is available to calculate the duplication rate without writing any output files. Since file I/O operations account for the majority of execution time, this mode is exceptionally fast and serves as a highly efficient feature for pipeline planning and data analyses.

## 3.1 : Algorithmic Complexity and Hardware-Level Optimizations

The architectural principal of `FastDedup`'s performance lies in shifting the deduplication problem from a string-matching paradigm to an integer-matching paradigm. By transforming raw paired-end sequences (e.g., 2 x 150 bytes) into a single 64-bit hash, reducing the data by 37.5 times in both space and time complexity.

### 3.1.1 Space Complexity and Memory Footprint

In a deterministic approach, storing a read pair requires retaining sequences of length $L$ (where $L = r_1 + r_2$). The space complexity for $N$ unique sequences scales linearly as $O(N \times L)$. For a standard 150 bp paired-end dataset, each pair consumes at least 300 bytes of raw character data, plus substantial data structure overhead (e.g., string pointers). Storing $10^8$ sequences typically exceeds 32 GB of RAM. By computing a 64-bit hash using `xxh3`, `FastDedup` reduce the sequence identity into exactly 8 bytes. This decouples the memory requirement from the read length, reducing the space complexity to $O(N)$ (since $8 \ll L$).

### 3.1.2 Time Complexity and Arithmetic Logic Unit (ALU) Efficiency

String-based deduplication necessitates sequential, byte-by-byte comparisons. In the worst-case scenario (ex: highly similar sequences or long identical prefixes), comparing two sequences requires $O(L)$ time. Conversely, while calculating the initial hash takes $O(L)$ time during the single-pass file reading, all subsequent identity verifications are reduced to integer operations. Comparing two 64-bit integers is executed natively by the CPU's ALU in exactly 1 clock cycle, effectively reducing the hash lookup and comparison time complexity to $O(1)$.

### 3.1.3 Cache Locality and Zero-Allocation Pipeline

Beyond theoretical Big-$O$ improvements, the 8-byte hash architecture heavily exploits modern CPU hardware design. Processors fetch data from RAM in discrete 64-byte cache lines. A 300-byte sequence spans multiple cache lines, leading to frequent cache misses and severe memory bandwidth bottlenecks. In contrast, exactly eight 64-bit hashes fit perfectly into a single cache line. As `fxhash` probes the internal hash set for collisions, it operates almost exclusively within the ultra-fast CPU cache (L1/L2), minimizing latency and maximizing throughput.

Furthermore, `FastDedup` processes byte slices directly from the `needletail` [18] parser into the `xxh3` hasher without ever allocating the raw string to the heap. This zero-allocation pipeline completely bypasses system-level memory allocation (`malloc`) overheads, ensuring that the execution time is strictly bound by disk I/O rather than RAM latency.

## 3.2 : The Mathematics of Dynamic Hashing

While `FDedup` defaults to the highly efficient 64-bit hashing strategy of `xxh3`, it dynamically assesses the risk of hash collisions to guarantee data integrity. The theoretical risk of a hash collision $p$ for $x$ sequences follows the standard Birthday Problem approximation [19,20]:

$$p = \frac{x^2}{2 * 2^N}$$

Where $N \in \{64, 128\}$ represents the bit-length of the hash. At the default 64-bit configuration and a probability threshold of $0.001$, it requires approximately $0.19 * 10^9$ sequences to reach a 1‰ collision risk. If the estimated dataset size pushes the collision probability beyond this threshold, `FDedup` automatically scales to 128-bit hashing, effectively nullifying the risk (requiring $0.28 * 10^{17}$ sequences to reach the same probability).

For paired-end reads, maintaining this theoretical probability requires bitwise operations. Let $S_1$ and $S_2$ represent the nucleotide sequences of Read 1 and Read 2, and $H$ be our non-cryptographic uniform hash function mapping to $\{0,1\}^N$. We define the independent hashes for each read as:

$$h_1 = H(S_1)$$

$$h_2 = H(S_2)$$

If the algorithm combined these hashes using a simple bitwise XOR ($\oplus$), a critical failure occurs when Read 1 and Read 2 are identical sequences ($S_1 = S_2$). In this case, $h_1 = h_2$, resulting in:

$$h_c = h_1 \oplus h_1 = 0$$

Every paired-end read with identical forward and reverse sequences would map to 0. This would cause massive, deterministic hash collisions, meaning the collision probability would no longer be bounded by the theoretical uniform distribution, but rather by the biological frequency of identical sequence pairs in the dataset.

To prevent symmetrical cancellation, the algorithm introduces a bitwise left-rotation by $\frac{N}{2}$ bits, denoted as $R(x, k)$. The unified fingerprint $h_u$ is instead calculated as:

$$h_u = h_1 \oplus R\left(h_2, \frac{N}{2}\right)$$

Under this operation, even if $S_1 = S_2$:

$$h_u = h_1 \oplus R\left(h_1, \frac{N}{2}\right) \neq 0$$

Because $R\left(h_1, \frac{N}{2}\right)$ shifts the lower half of the bits to the upper half and vice versa, $h_1 \oplus R\left(h_1, \frac{N}{2}\right)$ will almost never equal 0. It would only equal 0 if $h_1$ consisted of two perfectly identical halves, an event with an astronomically low probability of $2^{-\frac{N}{2}}$.

Since a bitwise rotation is a bijective function over the $N$-bit vector space, it perfectly preserves the original entropy of $h_2$. When two independent, uniformly distributed random variables are combined using a bitwise XOR, the resulting variable remains uniformly distributed. Therefore, the unified fingerprint $h_u$ safely maintains a strict uniform distribution, ensuring the overall collision risk remains uncorrupted and strictly bounded by the initial probability formula.

This is the mathematical foundation that allows `FDedup` to guarantee a collision probability of less than 1‰ for datasets containing up to approximately 190 million paired-end reads (380 million total sequences) when using 64-bit hashes.

## 3.3 : FxHash: Algorithmic Mechanics and ALU Optimizations

While `FastDedup` utilizes `xxh3` to compute sequence fingerprints, it relies on `fxhash` to manage these hashes within its internal memory structures. `fxhash` is a high-performance, non-cryptographic hashing algorithm derived from the internal hasher utilized in the Rust compiler (`rustc`) and Firefox. Its architectural strength lies not in collision resistance against deliberate Denial of Service (DoS) attacks, but rather in raw computational throughput and extremely low latency.

### 3.3.1 Word-Level Parallelism and Cache Locality

Unlike traditional algorithms such as FNV, which process data byte-by-byte, `fxhash` is engineered to aggressively consume data in native machine words. On 64-bit platforms, it processes 8 bytes (64 bits) per iteration. This word-level parallelism significantly reduces the number of loop iterations required, maximizing register utilization and minimizing memory fetches.

To maintain strict cache locality, `fxhash` reads directly from the byte slices using the CPU's native endianness without triggering any heap allocations. If the remaining data slice is smaller than 8 bytes, the algorithm dynamically cascades down to 4-byte, 2-byte, and 1-byte reads, ensuring every byte is processed safely.

### 3.3.2 The Mathematics of the State Transition

Let $H$ represent the internal state of the hash (initialized to 0), and $W$ represent an incoming data word. For every word extracted from the sequence, `fxhash` applies a highly optimized state transition function:

$$H_{\{i+1\}} = (R_L(H_i, 5) \oplus W_i) \times Kp \bmod \{2^N\}$$

Where:
- $N$ is the machine word size (typically 64 bits).
- $R_L(x, k)$ denotes a bitwise left-rotation of the state $x$ by $k$ bits (`ROTATE = 5`).
- $K$ is a predefined 64-bit magic constant or "seed": $K_{\{64\}} = 0\text{x}517\text{cc}1\text{b}727220\text{a}95$.

This specific sequence of operations is meticulously tailored for the Arithmetic Logic Unit (ALU) of modern CPUs. The bitwise left-rotation ($R_L$) ensures that previously hashed bits propagate rapidly across the state vector. Because a left-rotation is a bijective function, it perfectly preserves the entropy of $H_i$ while shifting bits to prevent localized clustering.

The bitwise XOR seamlessly introduces the new word $W_i$ into the state in a single CPU clock cycle. Finally, the wrapping multiplication by the large prime-like constant $K$ acts as an avalanche multiplier. It aggressively diffuses the newly mixed entropy across all 64 bits. The algorithm explicitly relies on modulo $2^{\{64\}}$ arithmetic (wrapping multiplication), leveraging the CPU's native handling of integer overflow without any computational penalty.

Through these hardware-level optimizations, `fxhash` dramatically reduces the time complexity of memory insertions and lookups, providing observable performance increases over byte-oriented hashers for any byte sequence with a length $ge5$.

### 3.3.3 Time and Space Complexity of FxHash

To fully appreciate the performance gains of `fxhash` within the architecture, we must evaluate its algorithmic complexity in contrast to traditional hash functions like FNV.

Let $L$ denote the length of the input byte sequence. A traditional byte-oriented hasher processes the sequence one byte at a time, requiring exactly $L$ state transitions, yielding a time complexity of $O(L)$ with a high hardware-level constant factor. Because `fxhash` aggressively consumes data in 8-byte (64-bit) chunks on 64-bit architectures, it reduces the number of required loop iterations to $l\lceil \frac{L}{8} r\lceil$. While the asymptotic time complexity remains mathematically $O(L)$ relative to the input length, the actual execution time is drastically reduced because the constant factor is effectively divided by eight.
Crucially, within the specific architecture of `FastDedup`, `fxhash` is not used to hash the raw, variable-length

biological sequences. Instead, it is employed by the `FxHashSet` to hash the 64-bit or 128-bit fingerprints already generated by `xxh3`. Consequently, the input length to `fxhash` is strictly fixed at 8 or 16 bytes ($L \in \{8, 16\}$). Because this input size is bounded and constant, the time complexity for computing the bucket routing index inside the hash set drops to a strict, deterministic $O(1)$.

The `fxhash` algorithm is exceptionally lightweight. It operates entirely in-place, maintaining only a single internal state variable (an 8-byte `usize` or `u64` on 64-bit platforms). It requires no auxiliary data structures, lookup tables, or intermediate buffers. Therefore, its auxiliary space complexity is strictly $O(1)$.

Furthermore, `fxhash` is designed to accept direct memory references (`&[u8]`) to the data slices. It processes the bytes without triggering any dynamic memory allocations on the heap. This zero-allocation property guarantees that the memory footprint remains absolutely flat during the hashing process, preventing memory fragmentation and avoiding the costly latency overhead of OS-level memory management (`malloc`/`free`).

### 3.4 : Architecture

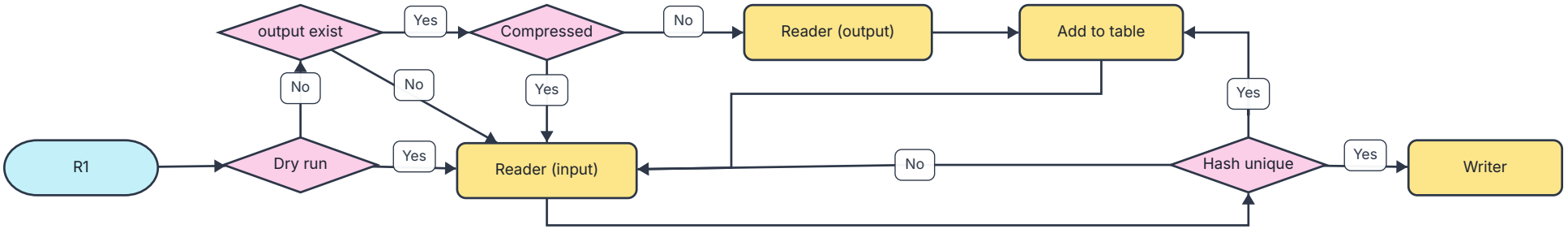



Figure 2: Architectural design of `FastDedup` for Single-End (SE) reads.

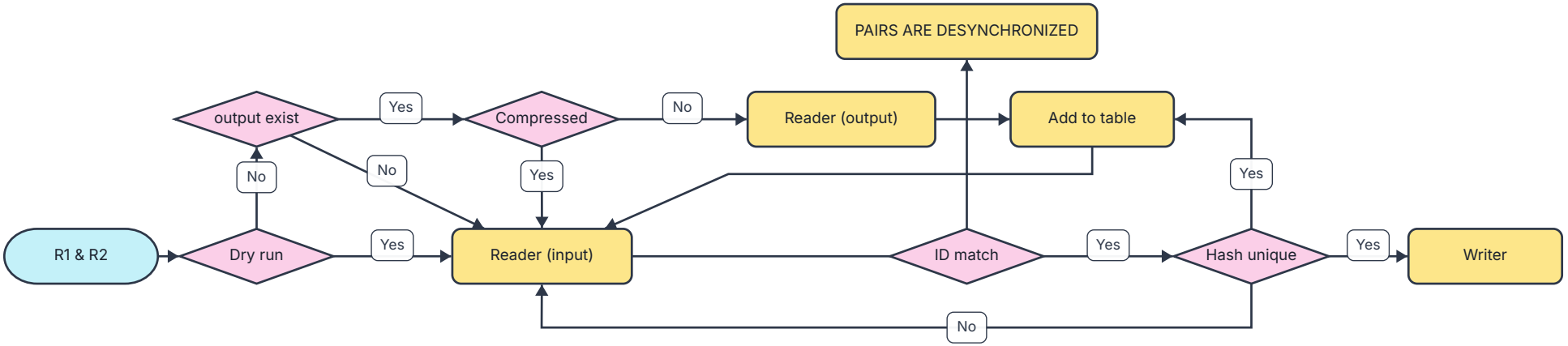



Figure 3: Architectural design of `FastDedup` for Paired-End (PE) reads.

# 4 – Benchmarking and performance evaluation

To test the performances of the differents tools evoked in the previous sections (Table 1), I benchmarked them on a series of synthetic datasets of varying sizes, ranging from 100,000 to 300 million paired-end reads to wich I added 10% of duplicates to each dataset. This allows us to evaluate the scalability of each tool and its suitability for different types of sequencing projects, from small-scale experiments to large population genomics studies. To run those benchamrks in a controled environment, I set-up a snakemake pipeline with each tool having 1 core and 33GB of RAM available. This pipeline ran on the High-Performance Computing (HPC) cluster I/O of Meso with AMD EPYC 9654 processors. If the tool goes above 32GB of RAM, it is killed and written as OOM (Out Of Memory) in the results table. In times where RAM and electricity are becoming increasingly costly and environmentally impactful, it is crucial to consider the resource efficiency of bioinformatics tools. Since those tools are for the first step for pre-treatment of data, it would be senseless to run them on luducrous ressources.

## 4.1 : Compressed Paired-End FASTQ

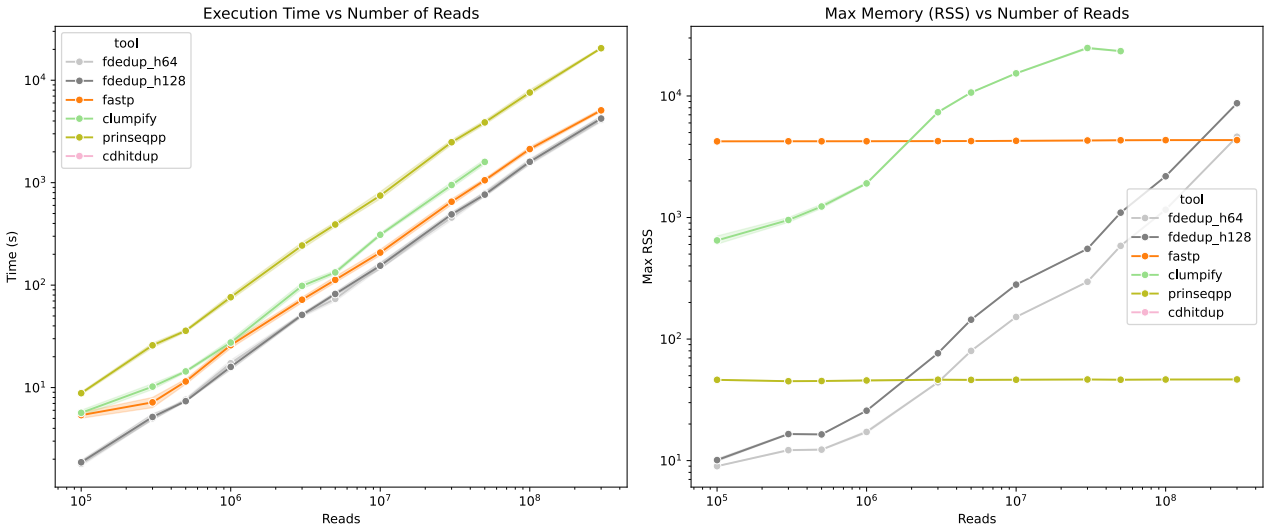



Figure 4: Execution time and memory usage across varying dataset sizes.

The default compression level of `fastp` is 4 but the `flate` library I use with `fdedup` is using a level 6 compression by default, which is more CPU intensive. Adding an option to set the compression level, we can now put `fastp` and `fdedup` on a more equal footing in terms of compression time. It would have been easier to just setp fastp to level 6, but it was a nice opportunity to add another feature to `fdedup`. In the end I added the option but I will rerun the benchmarks with `fastp` set to level 6 because all the other tools are level 6.

## 4.2 : Uncompressed Paired-End FASTQ

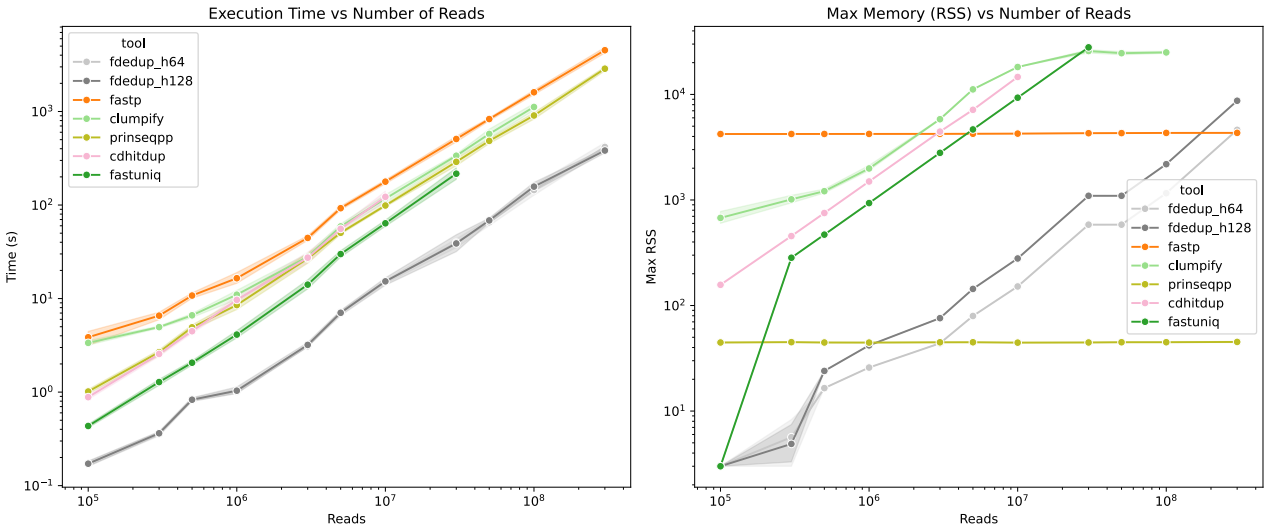



Figure 5: Execution time and memory usage across varying dataset sizes.

For uncompressed FASTQ file as inputs and uncompressed FASTQ file as outputs, `FastDedup` consistently outperformed all other tools in terms of execution time (Figure 5, Table 3). We can also observed that there are no real difference in execution time between the 64-bit and 128-bit hashing strategies, which is consistent with the theoretical analysis of their time complexity. We can also see that `FastDedup` takes up significantly less memory than all the other tools up until 5M reads, where `prinseq++` starts to use less memory than `FastDedup`.

However, starting at 30M reads, `CD-HIT-DUP` crashed due to an OOM error, followed by `fastuniq` at 50M reads, and `clumpify` at 300M reads. This left only `fastp`, `prinseq++` and `FastDedup` which was 6.4 times faster than `prinseq++` and 11.3 times faster than `fastp` at 300M reads, while using only 4.6GB and 8.7GB of RAM for 64-bit and 128-bit hashing respectively.

We can also see that the difference of RAM usage bewteen the 64-bit and 128-bit hashing strategies is doubled, which is consistent with the the fact that 64 times 2 is 128.

## 4.3 : Compressed in, decompressed out Paired-End FASTQ

Before talking about the results, we have to note that `fastuniq` is not included in this benchmark since it does not support compressed files. For `CD-HIT-DUP`, I got OOM errors for all read volumes. Since the tool doesn't output any log, I wasn't able to make it wrork for this specific benchmark.

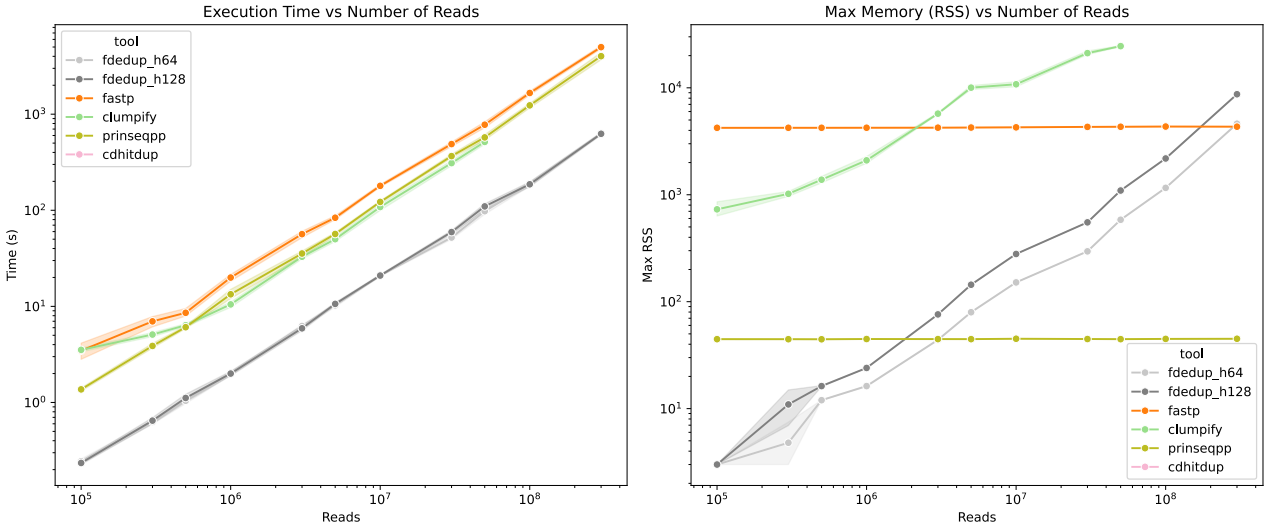



Figure 6: Execution time and memory usage across varying dataset sizes.

We observe very similar results as for uncompressed files (Figure 6, Table 4). `FastDedup` being the fastest tool and using the least amount of RAM up until 5M reads, where `prinseq++` starts to use less memory than `FastDedup`. We can also note very similar results for relative performance between each tools, which mean that all the tools scale in a similar way when the input files are compressed or not.

## 4.4 : Compressed Single-End FASTQ

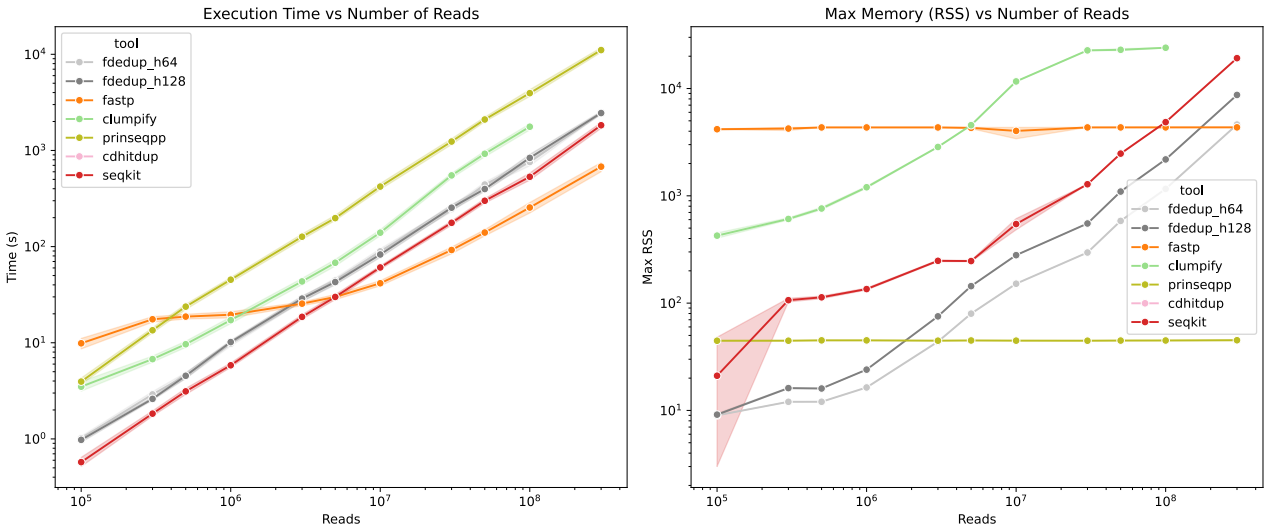



Figure 7: Execution time and memory usage across varying dataset sizes.

For SE data, we see 2 clear winners for execution time, `SeqKit` up until 3M reads and `fastp` for 5M reads and above (Figure 7, Table 5). Despite that `FastDedup` is having much better RAM efficiency than `SeqKit` and `fastp` while keeping a relatively good execution time, especially compared to `prinseq++` which is the slowest tool for SE compressed data.

## 4.5 : Decompressed Single-End FASTQ

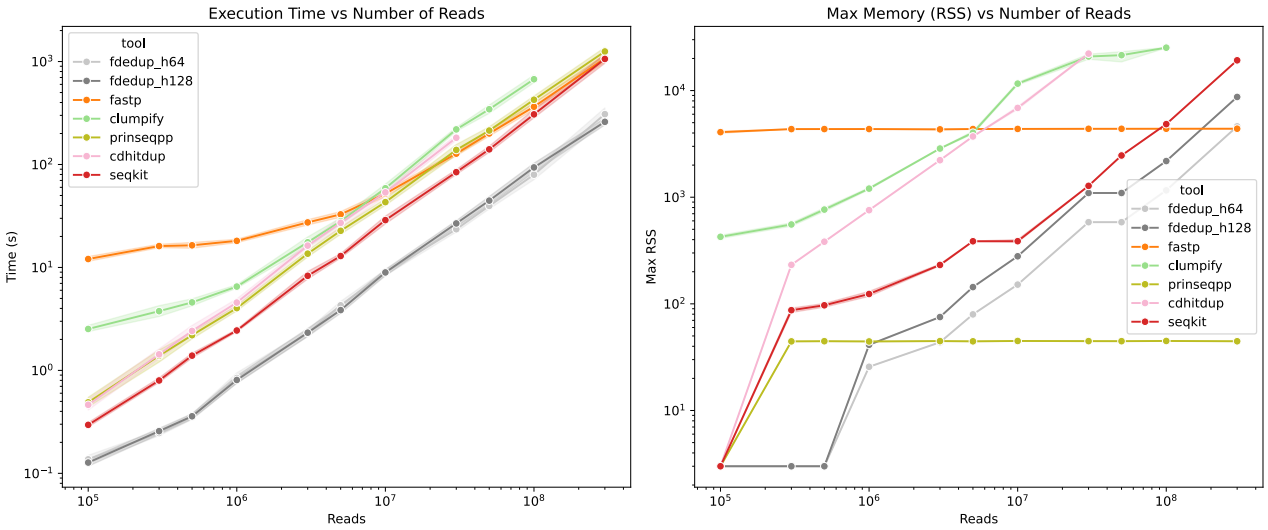



Figure 8: Execution time and memory usage across varying dataset sizes.

Here we get similar results as for uncompressed paired-end data, with `FastDedup` being the fastest tool and using the least amount of RAM up until 5M reads, where `prinseq++` starts to use less memory than `FastDedup` (Figure 8, Table 6).

## 4.6 : Compressed in, decompressed out Single-End FASTQ

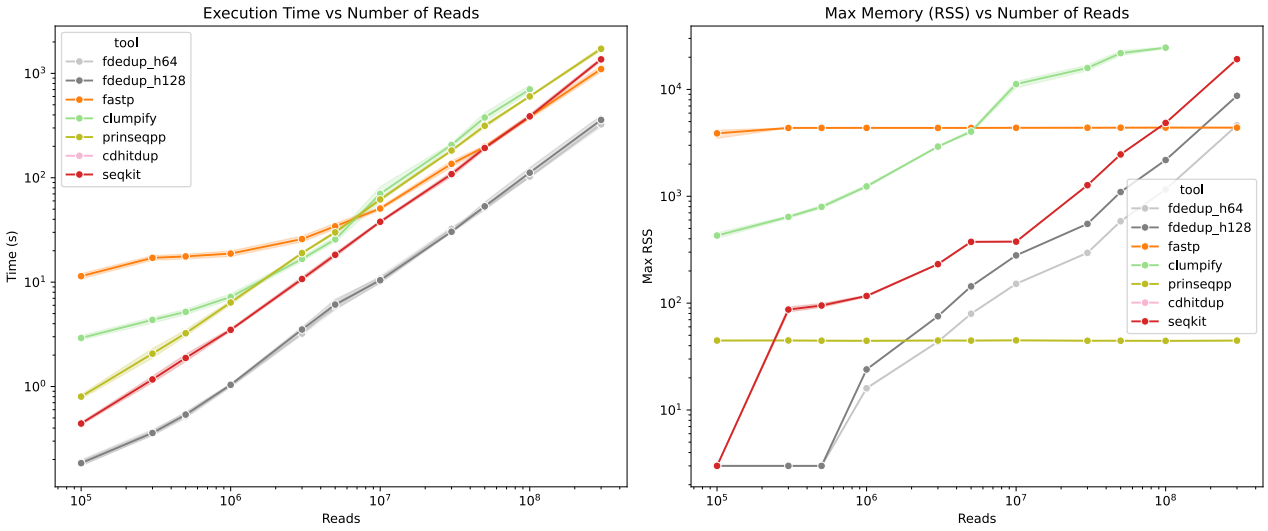



Figure 9: Execution time and memory usage across varying dataset sizes.

Well I fucked up the config of this one it is reruning right now.

# 5 – Discussion and future perspectives

Now that we have significant data from isolated benchmarks, we can discuss the implications of these results for real-world applications and the future development of `FastDedup`.

## 5.1 : Output Quality and Deduplication Accuracy

Despite each tool having very different approaches to deduplication, they all produced the pretty much the same output in terms of the number of unique sequences and the duplication rate. There is a small difference, but less than 0.1% between the tools. However, I wasn't able to estimate for `CD-HIT-DUP` since it doesn't output any log. Moreover, there is a problem with `prinseq++` starting at 10M reads where the duplication rate is higher than 10.1%, reaching up to more than 90% at 300M reads (Figure 10). Because of this we do not

recommend using `prinseq++` in your pipelines, since it starting to be more memory efficient than `FastDedup` at 5M reads, but starts having a very high false-positive rate at 10M reads.

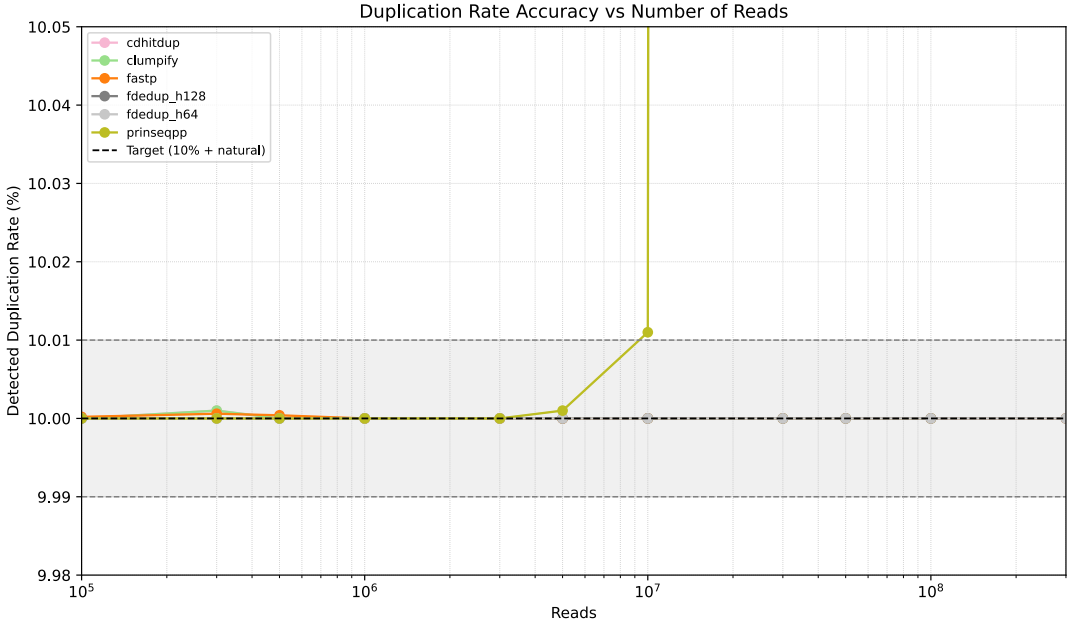



Figure 10: Execution time and memory usage across varying dataset sizes.

A similar pattern is observed for `clumpify` on signle-end data, but starting at 3M reads. It is above 10.1% at 10M reads but reaching around than 10.1% at 100M reads (Figure 11). This is still above the 0.1% false-positive threshold, but it is far better than the 90% false-positive rate of `prinseq++` at 300M reads.

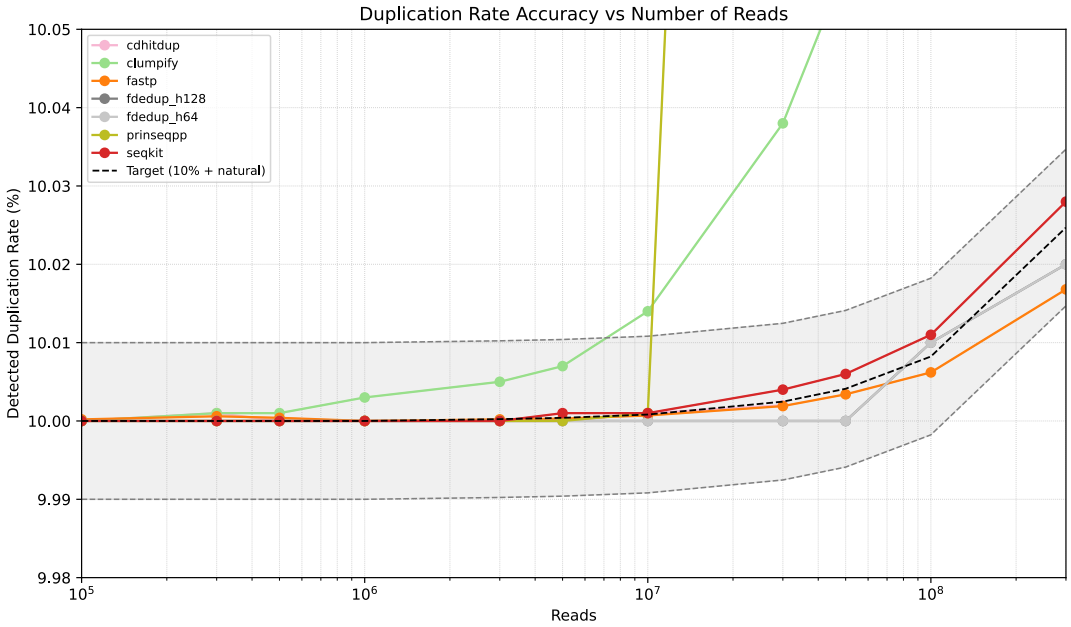



Figure 11: Execution time and memory usage across varying dataset sizes.

The target duplication value isn't constant across different volume because `wgsim` is a random read simulator. We can calculate the number of natural duplicate $C$ with $N$ randomly generate reads from a uniform distribution of $M$ possible start positions like $C \approx \frac{N^2}{2M}$. However, the human genome is not uniform, it contains high-frequency repeat families [21], so we have to aproximate $M$ with the number of unique start positions in the simulated dataset. We can aproximate $k = (2G)^{-1}$ using a known data point from your `wgsim` simulation. Looking at the generation of $270,000,000$ single-end reads, the simulation naturally produced roughly $60,000$ coincidental duplicates.

We can solve $k$ like:

$$60,000 = k \times (270,000,000)^2$$
$$k = \frac{60,000}{(270,000,000)^2}$$

We can simply now express the number of natural duplicates $C$ like $C = k \times N^2$ and add this $C$ to the 10% of duplicates we added to the dataset to get the expected duplication rate for each dataset size.

## 5.2 : Performance with Paired-End Reads

On all paired-end datasets, `FastDedup` outperformed all other tools in terms of execution time for all dataset sizes, while maintaining a very low memory footprint. However if the user is looking for the lowest possible memory usage, starting after 3M reads, `prinseq++` is the best choice for paired-end data, but it is more than 7 times slower than `FastDedup` for 5M reads and goes down to 6.4 times slower at 300M reads.

## 5.3 : Performance with Single-End Reads

## 5.4 : Performance with Compressed Files

## 5.5 : In short

| Tool | Algorithm / Approach | PE Support | Error Tolerance | Memory Usage | Key Limitations |
|---|---|---|---|---|---|
| `fastp` [3,4] | Probabilistic (Bloom filter/hashes) | Yes | No | Configurable | False-positive rate; long runtime |
| `FastUniq` [5] | Deterministic (Merge sort) | Yes | No | Very High | Severe RAM bottleneck; sensitive to input file order. |
| `Clumpify` [7] | k-mer grouping | Yes | Yes | Variable | Variable stability on massive datasets; high memory overhead. |
| `SeqKit rmdup` [8,9] | Exact matching (Hashing/MD5) | No | No | Very Low | Desynchronizes paired-end files. |
| `CD-HIT-DUP` [10,11] | Greedy incremental clustering | Yes | Yes | Very High | Stability issues in containerized environments. |
| `PRINSEQ++` [12] | Probabilistic (Bloom filter) | Yes | No | Low | False-positives rate |
| `FastDedup` | Exact matching (Hashing/xxh3) | Yes | No | Very Low | Theoretical negligible collision risk |

Table 1: Summary of existing tools for reads deduplication and their limitations.

### 5.6 : The Future of FastDedup

The current version of `FastDedup` is optimized for speed and memory efficiency, but there are some features that could further enhance its functionality. Firstly, the support for multithreading, which would allow the tool to take advantage of multiple CPU cores and further reduce execution time, especially for larger datasets. We can use thoses mutiple cores for parallelizing the file reading and hashing process, while maintaining a shared hash set for deduplication. This can however introduce some overhead due to thread synchronization, so it would require careful testing to ensure that the performance benefits outweigh the costs. However, those extra cores could be used for faster file compression and decompression. Indeed, writing a compressed file makes the writing phase taking more than 90% of the execution time.

Another feature that could be added is track of abundance, which would allow users to not only remove duplicates but also to quantify how many times each unique sequence was observed in the input dataset. This can be particularly useful for applications like metagenomics or transcriptomics, where the abundance of certain sequences can provide important biological insights.

## 6 – Conclusion and Recommendations

`FastDedup` represents a significant advancement for FASTX deduplication, offering unparalleled speed and memory efficiency compared to existing tools. Its innovative use of non-cryptographic hashing and dynamic scaling strategies allows it to handle massive datasets with ease, making it an ideal choice for modern high-throughput sequencing projects. While other tools may offer specific features or error tolerance, `FastDedup` provides a robust, scalable solution for bioinformaticians seeking to optimize their data processing pipelines without sacrificing accuracy or resource efficiency. For paired-end datasets, `FastDedup` is the clear choice for users prioritizing speed. For single-end datasets, TODO.

We recommand using `FastDedup` at the beginning of the data processing pipeline, immediately after quality control and adapter trimming steps, to ensure that downstream analyses are not biased by PCR duplicates. By using uncompressed output files, users can further maximize the performance benefits of `FastDedup`, and then deleting the uncompressed files after further processing to save storage space.

This software is protected by the MIT License, and you can find it on:
- GitHub https://github.com/RaphaelRibes/FDedup.git
- Bionconda https://anaconda.org/bioconda/fdedup
- Cargo https://crates.io/crates/fastdedup

## 7 – Acknowledgements

This paper was written by Raphaël Ribes, reviewed and edited by Céline Mandier, with the assistance of Gemini 3.1 for grammar, and language correction. Code architecture and design was developed by Raphaël Ribes, with autocompletion assisted by Gemini 3.1. Feedback on the algorithm design and ideas for features were provided by Céline Mandier and Alban Mancheron, the implementation was done by Raphaël Ribes. The testsuite was mostly developped Claude Sonnet 4.6, reviewed by Raphaël Ribes.

Computations were performed on the ISDM-MESO HPC platform, funded in the framework of State-region planning contracts (Contrat de plan État-région – CPER) by the French Government, the Occitanie/Pyrénées-Méditerranée Region, Montpellier Méditerranée Métropole, and the University of Montpellier.

# Bibliography


[1] M. T. W. Ebbert, M. E. Wadsworth, L. A. Staley, K. L. Hoyt, B. Pickett, J. Miller, J. Duce, J. S. K. Kauwe, and P. G. Ridge, Evaluating the necessity of PCR duplicate removal from next-generation sequencing data and a comparison of approaches, BMC Bioinformatics **17**, 239 (2016).

[2] Z. Zhang, L. Zhang, G. Zhang, Z. Zhao, H. Wang, and F. Ju, Deduplication Improves Cost-Efficiency and Yields of De Novo Assembly and Binning of Shotgun Metagenomes in Microbiome Research, Spectrum **11**, e4282 (2023).

[3] S. Chen, Y. Zhou, Y. Chen, and J. Gu, fastp: an ultra-fast all-in-one FASTQ preprocessor, Bioinformatics **34**, i884 (2018).

[4] S. Chen, Ultrafast one-pass FASTQ data preprocessing, quality control, and deduplication using fastp, Imeta **2**, e107 (2023).

[5] H. Xu, X. Luo, J. Qian, X. Pang, J. Song, G. Qian, J. Chen, and S. Chen, FastUniq: A Fast De Novo Duplicates Removal Tool for Paired Short Reads, Plos ONE **7**, e52249 (2012).

[6] A. I. Sigorskikh, M. A. Kompaniets, I. S. Ilnitskiy, G. K. Ryabykh, and A. A. Mironov, Fastq-dupaway: a fast and memory-efficient tool for deduplication of single- and paired-end NGS data, Scientific Reports **15**, 45303 (2025).

[7] B. Bushnell, *Bbmap: A Fast, Accurate, Splice-Aware Aligner (And Bbtools)*, https://archive.jgi.doe.gov/data-and-tools/software-tools/bbtools/.

[8] W. Shen, S. Le, Y. Li, and F. Hu, SeqKit: A Cross-Platform and Ultrafast Toolkit for FASTA/Q File Manipulation, Plos ONE **11**, e163962 (2016).

[9] W. Shen, B. Sipos, and L. Zhao, SeqKit2: A Swiss Army Knife for Sequence and Alignment Processing, Imeta **3**, e191 (2024).

[10] L. Fu, B. Niu, Z. Zhu, S. Wu, and W. Li, CD-HIT: accelerated for clustering the next-generation sequencing data, Bioinformatics **28**, 3150 (2012).

[11] Y. Huang, B. Niu, Y. Gao, L. Fu, and W. Li, CD-HIT Suite: a web server for clustering and comparing biological sequences, Bioinformatics **26**, 680 (2010).

[12] V. A. Cantu, J. Sadural, and R. Edwards, PRINSEQ++, a multi-threaded tool for fast and efficient quality control and preprocessing of sequencing datasets, Peerj Preprints **7**, e27553v1 (2019).

[13] H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, and R. Durbin, The sequence alignment/map format and SAMtools, Bioinformatics **25**, 2078 (2009).

[14] Broad Institute, *Picard Toolkit*, http://broadinstitute.github.io/picard/.

[15] Y. Collet, *Xxhash: Extremely Fast Non-Cryptographic Hash Algorithm*, https://github.com/Cyan4973/xxHash.

[16] DoumanAsh, *Xxhash-Rust: Rust Implementation of Xxhash*, https://github.com/DoumanAsh/xxhash-rust.

[17] cbreeden, *Fxhash: A Fast, Non-Secure, Hashing Algorithm Derived from an Internal Hasher in Firefox*, https://github.com/cbreeden/fxhash.

[18] M. Gore, *Needletail: Fast FASTX Parsing and K-Mer Methods in Rust*, https://github.com/onecodex/needletail.

[19] W. Feller, *An Introduction to Probability Theory and Its Applications*, 3rd ed., Vol. 1 (John Wiley & Sons, 1968).

[20] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. (The MIT Press, 2009).

[21] T. J. Treangen and S. L. Salzberg, Repetitive DNA and next-generation sequencing: computational challenges and solutions, Nature Reviews Genetics **13**, 36 (2012).

# 8 – Annexes

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM |
| `clumpify` | 5.7±0.6<br>648±84 | 10.2±1.0<br>954±72 | 14.4±0.6<br>1232±78 | 27.5±2.7<br>1909±48 | 98.3±10.6<br>7351±122 | 133±9.1<br>10671±26 | 310±18.2<br>15352±128 | 949±102<br>24814±672 | 1595±145<br>23383±133 | OOM | OOM |
| `fastp` | 5.4±0.6<br>4233±1 | 7.2±1.3<br>4238±1 | 11.5±1.1<br>4238±2 | 25.9±2.8<br>4240±2 | 72.0±7.7<br>4252±6 | 113±12.8<br>4257±9 | 208±22.6<br>4275±18 | 652±68.4<br>4303±22 | 1056±76.7<br>4322±31 | 2128±163<br>4333±20 | 5083±252<br>4336±17 |
| `fdedup_h128` | 1.9±0.1<br>10±0 | 5.2±0.5<br>17±0 | 7.4±0.2<br>16±0 | 15.9±1.6<br>26±0 | 51.2±2.9<br>76±0 | 82.0±4.6<br>144±0 | 155±18.0<br>281±0 | 490±43.7<br>552±0 | 765±59.7<br>1097±0 | 1599±130<br>2184±0 | 4226±410<br>8713±0 |
| `fdedup_h64` | 1.8±0.1<br>9±0 | 5.0±0.2<br>12±0 | 7.6±0.4<br>12±0 | 17.3±1.9<br>17±1 | 50.7±3.2<br>44±0 | 73.4±6.9<br>80±0 | 154±14.8<br>152±1 | 457±48.9<br>296±1 | 782±66.9<br>585±1 | 1599±188<br>1160±1 | 4101±387<br>4617±0 |
| `prinseqpp` | 8.8±0.5<br>46±1 | 25.9±2.1<br>45±0 | 35.8±1.8<br>45±0 | 76.2±7.0<br>46±1 | 244±28.1<br>46±0 | 391±36.0<br>46±1 | 748±99.2<br>46±0 | 2482±222<br>47±0 | 3875±294<br>46±0 | 7584±720<br>46±0 | 20538±720<br>47±0 |

Table 2: Execution time and memory usage across varying dataset sizes for compressed input and compressed output paired-end FASTQ files.

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | 0.9±0.1<br>157±1 | 2.6±0.2<br>455±3 | 4.5±0.3<br>752±0 | 9.7±1.8<br>1497±1 | 27.4±6.0<br>4443±1 | 55.6±5.1<br>7146±118 | 122±26.8<br>14639±1 | OOM | OOM | OOM | OOM |
| `clumpify` | 3.4±0.3<br>676±155 | 5.0±0.2<br>1013±131 | 6.6±0.5<br>1211±85 | 11.0±2.1<br>1989±181 | 28.0±3.8<br>5825±83 | 59.1±6.0<br>11166±35 | 117±17.6<br>18191±21 | 337±19.9<br>25774±1431 | 576±105<br>24599±1394 | 1118±147<br>25008±829 | OOM |
| `fastp` | 3.9±1.1<br>4217±1 | 6.6±0.9<br>4219±1 | 10.8±1.1<br>4222±1 | 16.5±3.8<br>4225±3 | 44.5±4.1<br>4230±3 | 92.5±7.5<br>4239±7 | 178±14.0<br>4254±10 | 508±54.2<br>4292±15 | 831±50.3<br>4302±17 | 1606±149<br>4319±24 | 4522±527<br>4323±18 |
| `fastuniq` | 0.4±0.0<br>3±0 | 1.3±0.1<br>284±0 | 2.1±0.2<br>470±0 | 4.1±0.6<br>935±0 | 14.1±2.7<br>2796±0 | 29.9±3.8<br>4658±0 | 63.8±8.0<br>9312±0 | 216±51.7<br>27927±0 | OOM | OOM | OOM |
| `fdedup_h128` | 0.2±0.0<br>3±0 | 0.4±0.0<br>5±4 | 0.8±0.1<br>24±0 | 1.0±0.1<br>42±0 | 3.2±0.3<br>76±0 | 7.0±0.8<br>144±0 | 15.3±2.2<br>279±0 | 38.8±13.8<br>1096±1 | 68.5±6.8<br>1095±0 | 157±26.9<br>2184±1 | 383±32.8<br>8712±0 |
| `fdedup_h64` | 0.2±0.0<br>3±0 | 0.4±0.0<br>6±4 | 0.8±0.1<br>17±0 | 1.0±0.1<br>26±0 | 3.1±0.1<br>44±0 | 7.2±0.4<br>80±0 | 15.1±1.1<br>152±0 | 39.7±6.6<br>584±0 | 65.5±8.9<br>584±0 | 145±29.5<br>1160±0 | 418±65.7<br>4616±0 |
| `prinseqpp` | 1.0±0.1<br>45±0 | 2.7±0.3<br>45±0 | 4.9±0.5<br>45±1 | 8.5±1.6<br>45±0 | 26.8±5.3<br>45±0 | 50.5±2.9<br>45±0 | 98.8±5.6<br>45±0 | 290±43.9<br>45±0 | 485±45.8<br>45±0 | 907±117<br>45±0 | 2869±232<br>45±1 |

Table 3: Execution time and memory usage across varying dataset sizes for uncompressed input and uncompressed output paired-end FASTQ files.

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM |
| `clumpify` | 3.5±0.3<br>730±189 | 5.1±0.4<br>1020±89 | 6.4±0.4<br>1383±162 | 10.5±1.1<br>2098±240 | 32.8±3.0<br>5735±134 | 50.0±4.2<br>10040±669 | 108±13.6<br>10794±950 | 310±31.5<br>21110±1336 | 515±46.4<br>24537±428 | OOM | OOM |
| `fastp` | 3.5±1.1<br>4226±1 | 7.0±1.4<br>4230±1 | 8.6±1.3<br>4230±1 | 20.0±2.6<br>4233±1 | 56.5±6.7<br>4240±3 | 83.7±6.4<br>4251±6 | 180±9.1<br>4270±15 | 489±42.6<br>4312±25 | 777±94.3<br>4324±23 | 1662±121<br>4344±13 | 4986±304<br>4332±12 |
| `fdedup_h128` | 0.2±0.0<br>3±0 | 0.6±0.1<br>11±6 | 1.1±0.2<br>16±0 | 2.0±0.2<br>24±0 | 5.9±0.5<br>76±0 | 10.6±0.6<br>144±0 | 20.9±0.8<br>279±1 | 59.4±4.1<br>552±0 | 110±12.9<br>1095±0 | 186±17.1<br>2184±0 | 625±26.8<br>8712±0 |
| `fdedup_h64` | 0.2±0.0<br>3±0 | 0.6±0.1<br>5±4 | 1.0±0.1<br>12±0 | 2.0±0.1<br>16±0 | 6.2±0.4<br>44±0 | 10.2±0.6<br>80±0 | 21.0±0.8<br>151±0 | 52.1±3.8<br>296±0 | 97.9±10.5<br>584±0 | 188±15.7<br>1160±0 | 629±24.8<br>4616±0 |
| `prinseqpp` | 1.4±0.1<br>45±1 | 3.9±0.4<br>45±0 | 6.1±0.3<br>45±0 | 13.4±2.4<br>45±0 | 35.5±4.1<br>45±0 | 56.7±3.3<br>45±0 | 122±5.6<br>45±0 | 366±25.5<br>45±0 | 573±59.4<br>45±0 | 1235±112<br>45±0 | 4015±592<br>45±0 |

Table 4: Execution time and memory usage across varying dataset sizes for compressed input and uncompressed output paired-end FASTQ files.

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM |
| `clumpify` | 3.5±0.6<br>426±40 | 6.8±0.8<br>609±24 | 9.7±1.2<br>762±34 | 17.2±2.2<br>1202±42 | 43.4±5.4<br>2853±21 | 68.0±8.4<br>4556±43 | 140±16.4<br>11646±53 | 550±52.8<br>22673±169 | 924±103<br>22955±803 | 1765±216<br>23990±87 | OOM |
| `fastp` | 9.9±2.1<br>4177±67 | 17.6±1.8<br>4235±257 | 18.7±1.8<br>4339±1 | 19.6±2.3<br>4338±2 | 25.5±1.8<br>4338±1 | 29.9±3.1<br>4300±110 | 41.6±4.2<br>4027±938 | 92.2±12.3<br>4339±1 | 140±16.6<br>4341±3 | 255±52.2<br>4342±8 | 678±115<br>4346±10 |
| `fdedup_h128` | 1.0±0.1<br>9±0 | 2.6±0.1<br>16±0 | 4.5±0.5<br>16±0 | 10.2±0.6<br>24±0 | 28.8±2.9<br>75±0 | 42.8±4.8<br>144±0 | 82.8±10.1<br>280±0 | 254±28.8<br>552±0 | 397±40.2<br>1096±1 | 837±123<br>2184±0 | 2450±143<br>8712±0 |
| `fdedup_h64` | 1.0±0.1<br>9±0 | 2.9±0.3<br>12±0 | 4.5±0.5<br>12±0 | 9.8±0.9<br>16±0 | 28.4±2.4<br>44±0 | 44.3±5.0<br>80±0 | 89.2±10.1<br>152±0 | 246±25.0<br>296±0 | 442±40.2<br>584±0 | 759±121<br>1160±0 | 2457±97.3<br>4616±0 |
| `prinseqpp` | 3.9±0.5<br>45±0 | 13.5±1.4<br>45±0 | 23.7±2.4<br>45±0 | 45.3±4.3<br>45±0 | 127±14.3<br>45±0 | 198±20.4<br>45±0 | 421±49.9<br>45±0 | 1238±140<br>45±0 | 2099±194<br>45±0 | 3944±483<br>45±0 | 11065±1007<br>45±1 |
| `seqkit` | 0.6±0.1<br>21±36 | 1.8±0.2<br>107±6 | 3.1±0.4<br>113±5 | 5.8±0.5<br>135±5 | 18.6±1.7<br>248±6 | 30.0±2.9<br>247±5 | 60.7±4.2<br>546±107 | 177±15.3<br>1282±4 | 300±26.3<br>2474±6 | 532±73.3<br>4862±7 | 1828±201<br>19189±16 |

Table 5: Execution time and memory usage across varying dataset sizes for compressed input and compressed output single-end FASTQ files.

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | 0.5±0.1<br>3±0 | 1.4±0.2<br>232±0 | 2.4±0.5<br>383±0 | 4.6±0.6<br>756±1 | 16.3±1.4<br>2224±1 | 27.1±5.3<br>3713±12 | 53.6±7.7<br>6875±367 | 182±17.2<br>22202±220 | OOM | OOM | OOM |
| `clumpify` | 2.5±0.3<br>425±25 | 3.8±0.7<br>555±33 | 4.6±0.6<br>766±51 | 6.5±0.4<br>1204±46 | 17.6±3.2<br>2861±21 | 28.1±3.3<br>4024±226 | 58.8±9.2<br>11618±520 | 219±15.9<br>20868±1776 | 344±52.6<br>21473±4147 | 673±92.0<br>25250±290 | OOM |
| `fastp` | 12.1±1.1<br>4075±149 | 16.1±1.0<br>4352±1 | 16.4±1.7<br>4356±2 | 18.1±1.1<br>4357±4 | 27.4±2.6<br>4328±69 | 32.8±3.4<br>4359±6 | 52.0±5.9<br>4368±12 | 127±9.3<br>4380±9 | 200±19.8<br>4381±9 | 364±45.2<br>4385±6 | 1074±146<br>4389±2 |
| `fdedup_h128` | 0.1±0.0<br>3±0 | 0.3±0.0<br>3±0 | 0.4±0.0<br>3±0 | 0.8±0.1<br>41±1 | 2.3±0.4<br>75±0 | 3.9±0.5<br>144±1 | 8.9±1.0<br>279±0 | 26.8±3.0<br>1095±0 | 44.6±7.0<br>1095±0 | 93.7±15.2<br>2183±1 | 260±34.3<br>8711±0 |
| `fdedup_h64` | 0.1±0.0<br>3±0 | 0.2±0.0<br>3±0 | 0.4±0.0<br>3±0 | 0.8±0.2<br>26±0 | 2.3±0.3<br>44±0 | 4.3±0.6<br>80±0 | 9.0±0.9<br>152±0 | 23.5±3.0<br>583±0 | 39.6±4.5<br>583±0 | 79.7±15.5<br>1160±0 | 308±76.0<br>4615±0 |
| `prinseqpp` | 0.5±0.1<br>3±0 | 1.4±0.3<br>44±0 | 2.2±0.3<br>45±0 | 4.0±0.4<br>44±0 | 13.6±1.7<br>45±0 | 22.7±3.0<br>45±1 | 43.1±5.3<br>45±0 | 139±27.3<br>45±1 | 214±30.6<br>45±0 | 426±54.3<br>45±0 | 1252±205<br>45±0 |
| `seqkit` | 0.3±0.0<br>3±0 | 0.8±0.1<br>87±7 | 1.4±0.1<br>97±7 | 2.4±0.1<br>124±11 | 8.3±1.2<br>232±8 | 12.9±1.0<br>386±3 | 28.8±4.0<br>387±20 | 84.3±7.7<br>1278±7 | 140±16.5<br>2461±6 | 307±48.0<br>4847±14 | 1060±183<br>19215±23 |

Table 6: Execution time and memory usage across varying dataset sizes for uncompressed input and uncompressed output single-end FASTQ files.

| **Tool**<br>*(s)*<br>*(MB)* | **0.1M** | **0.3M** | **0.5M** | **1.0M** | **3.0M** | **5.0M** | **10.0M** | **30.0M** | **50.0M** | **100.0M** | **300.0M** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| `cdhitdup` | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM | OOM |
| `clumpify` | 2.9±0.2<br>429±42 | 4.4±0.5<br>642±29 | 5.2±0.5<br>796±48 | 7.2±0.8<br>1237±61 | 16.7±1.4<br>2917±37 | 25.7±2.4<br>4032±226 | 70.2±19.3<br>11215±1441 | 206±12.9<br>15876±1613 | 378±62.0<br>21841±1844 | 703±98.9<br>24580±680 | OOM |
| `fastp` | 11.4±1.1<br>3879±586 | 17.1±1.4<br>4359±7 | 17.6±1.5<br>4365±4 | 18.8±1.9<br>4367±2 | 25.9±3.0<br>4362±4 | 34.3±3.7<br>4363±4 | 50.7±2.9<br>4374±10 | 136±15.2<br>4378±8 | 198±10.3<br>4386±15 | 383±27.6<br>4392±7 | 1099±120<br>4390±9 |
| `fdedup_h128` | 0.2±0.0<br>3±0 | 0.4±0.0<br>3±0 | 0.5±0.1<br>3±0 | 1.0±0.0<br>24±0 | 3.5±0.3<br>75±0 | 6.1±1.1<br>144±1 | 10.4±1.1<br>280±0 | 30.4±1.1<br>552±0 | 53.2±7.4<br>1096±0 | 112±19.5<br>2184±1 | 358±55.5<br>8712±0 |
| `fdedup_h64` | 0.2±0.0<br>3±0 | 0.4±0.0<br>3±0 | 0.5±0.0<br>3±0 | 1.0±0.1<br>16±0 | 3.2±0.1<br>44±0 | 5.9±1.0<br>80±0 | 10.7±1.2<br>152±0 | 32.4±2.4<br>296±0 | 51.8±1.6<br>584±0 | 103±4.4<br>1160±0 | 324±30.3<br>4615±0 |
| `prinseqpp` | 0.8±0.1<br>45±0 | 2.1±0.4<br>45±0 | 3.2±0.4<br>45±0 | 6.4±0.5<br>44±0 | 19.0±0.5<br>45±0 | 30.0±1.6<br>45±0 | 62.0±3.3<br>45±0 | 182±7.0<br>44±0 | 315±23.6<br>44±0 | 602±14.2<br>44±0 | 1721±118<br>45±0 |
| `seqkit` | 0.4±0.0<br>3±0 | 1.2±0.1<br>87±7 | 1.9±0.3<br>95±6 | 3.5±0.2<br>117±4 | 10.7±0.7<br>232±4 | 18.3±1.3<br>375±6 | 37.9±1.5<br>376±6 | 108±9.4<br>1270±2 | 193±13.0<br>2464±5 | 387±25.1<br>4856±8 | 1362±87.0<br>19189±24 |

Table 7: Execution time and memory usage across varying dataset sizes for compressed input and uncompressed output single-end FASTQ files.